

Pandas DataFrame Methods

INSPECTION & EXPLORATION

head(n=5) – Returns the first n rows.

python

df.head(3)

tail(n=5) – Returns the last n rows.

python

df.tail(3)

info() – Prints column names, dtypes, non-null counts.

python

df.info()

describe() – Descriptive statistics for numeric columns.

python

df.describe()

memory_usage() – Memory used by each column in bytes.

python

df.memory_usage(deep=True)



COPYING & STRUCTURE

copy() – Deep copy of the DataFrame.

python

df2 = df.copy()

transpose() – Flip rows and columns.

python

```
df.transpose() # or df.T
```

DROPPING & CLEANING

drop() – Remove rows or columns.

python

```
df.drop(columns=['Age']) # drop column
```

```
df.drop(index=[0, 1]) # drop rows by index
```

drop_duplicates() – Remove duplicate rows.

python

```
df.drop_duplicates()
```

```
df.drop_duplicates(subset=['Name']) # based on one column
```

uplicated() – Boolean mask of duplicate rows.

python

```
df.duplicated()
```

```
df[df.duplicated()] # show duplicate rows
```

RENAMING & TYPE CONVERSION

rename() – Rename columns or index labels.

python

```
df.rename(columns={'Name': 'Full Name', 'Age': 'Years'})
```

astype() – Cast column to a different data type.

python

```
df['Age'] = df['Age'].astype(float)
df['Score'] = df['Score'].astype(str)
```

MISSING VALUE HANDLING

fillna() – Fill missing values with a value or method.

python

```
df.fillna(0)          # fill all NaN with 0
df['Age'].fillna(df['Age'].mean()) # fill with mean
df.fillna(method='ffill') # forward fill
```

dropna() – Drop rows/columns with missing values.

python

```
df.dropna()          # drop rows with any NaN
df.dropna(subset=['Age']) # drop only if Age is NaN
df.dropna(axis=1)      # drop columns with NaN
```

isnull() – Boolean mask where values are NaN.

python

```
df.isnull()
df.isnull().sum()      # count NaNs per column
```

notnull() – Boolean mask where values are NOT NaN.

python

```
df.notnull()
df[df['Age'].notnull()] # rows where Age is not NaN
```

REPLACING VALUES

replace() – Replace specific values across the DataFrame.

```
python
df.replace(0, 999)
df.replace({'Gender': {'M': 'Male', 'F': 'Female'}})
```

SORTING

sort_values() – Sort by column values.

```
python
df.sort_values('Age')
df.sort_values('Age', ascending=False)
df.sort_values(['Dept', 'Salary'], ascending=[True, False])
```

sort_index() – Sort by index.

```
python
df.sort_index()
df.sort_index(ascending=False)
```

GROUPING & AGGREGATION

groupby() – Group rows by column values.

```
python
df.groupby('Dept')['Salary'].mean()
df.groupby(['Dept', 'Gender'])['Salary'].sum()
```

pivot_table() – Summarize data like an Excel pivot table.

```
python
df.pivot_table(values='Salary', index='Dept', aggfunc='mean')
df.pivot_table(values='Sales', index='Region', columns='Month', aggfunc='sum')
```

agg() – Apply multiple aggregation functions at once.

python

```
df.groupby('Dept').agg({'Salary': ['mean', 'max'], 'Age': 'min'})
```

```
df['Salary'].agg(['mean', 'std', 'max'])
```

transform() – Apply function but keep original DataFrame shape.

python

```
df['Salary_Norm'] = df.groupby('Dept')['Salary'].transform(lambda x: (x - x.mean()) / x.std())
```

COMBINING DataFrames

merge() – SQL-style join on columns.

python

```
pd.merge(df1, df2, on='ID')
```

```
pd.merge(df1, df2, on='ID', how='left')
```

```
pd.merge(df1, df2, left_on='emp_id', right_on='id', how='inner')
```

join() – Join on index.

python

```
df1.join(df2)
```

```
df1.join(df2, how='outer')
```

concat() – Stack DataFrames vertically or horizontally.

python

```
pd.concat([df1, df2])          # stack rows
```

```
pd.concat([df1, df2], axis=1)  # stack columns
```

```
pd.concat([df1, df2], ignore_index=True)
```

FILTERING & QUERYING

query() – Filter rows using a string expression.

python

```
df.query('Age > 30')
```

```
df.query('Dept == "HR" and Salary > 50000')
```

```
df.query('Age in [25, 30, 35]')
```

where() – Keep values where condition is True, else replace.

python

```
df.where(df['Age'] > 25)      # NaN where condition is False
```

```
df['Salary'].where(df['Salary'] > 0, other=0)
```

mask() – Opposite of where; replace where condition is True.

python

```
df.mask(df['Age'] < 18)      # NaN where condition is True
```

```
df['Score'].mask(df['Score'] < 0, other=0)
```

clip() – Limit values to a min/max range.

python

```
df['Score'].clip(0, 100)     # values clamped between 0 and 100
```

```
df.clip(lower=0, upper=500)
```

APPLYING FUNCTIONS

apply() – Apply a function along rows or columns.

python

```
df['Salary'].apply(lambda x: x * 1.1)    # 10% raise
```

```
df.apply(lambda col: col.max() - col.min()) # range of each column
```

applymap() – Apply a function element-wise to entire DataFrame.

python

```
df.applymap(str)                # convert all to string
df.applymap(lambda x: round(x, 2))  # round all values
```

map() – Element-wise mapping on a Series.

python

```
df['Gender'].map({'M': 'Male', 'F': 'Female'})
df['Score'].map(lambda x: 'Pass' if x >= 50 else 'Fail')
```

VALUE ANALYSIS

value_counts() – Count occurrences of unique values.

python

```
df['Dept'].value_counts()
df['Dept'].value_counts(normalize=True)  # as proportions
```

unique() – Return unique values in a Series.

python

```
df['Dept'].unique()  # array of unique values
```

nunique() – Count number of unique values.

python

```
df.nunique()  # unique count per column
df['Dept'].nunique()
```

sample() – Return a random sample of rows.

python

```
df.sample(5)  # 5 random rows
df.sample(frac=0.1)  # 10% of rows
df.sample(5, random_state=42)  # reproducible sample
```

INDEX OPERATIONS

set_index() – Set a column as the index.

python

```
df.set_index('ID')
```

```
df.set_index('Name', inplace=True)
```

reset_index() – Reset index back to default integer index.

python

```
df.reset_index()
```

```
df.reset_index(drop=True)    # discard old index
```

reindex() – Conform DataFrame to a new index.

python

```
df.reindex([0, 1, 2, 10, 20])  # adds NaN for missing indices
```

```
df.reindex(columns=['Name', 'Age', 'NewCol'])
```

INSERTING COLUMNS

insert() – Insert a column at a specific position.

python

```
df.insert(2, 'Bonus', df['Salary'] * 0.1)  # insert at position 2
```

STATISTICS & MATH

mean() – Average of values.

python

```
df['Salary'].mean()
```

```
df.mean()          # mean of all numeric columns
```

median() – Middle value.

python

```
df['Age'].median()
```

mode() – Most frequently occurring value.

python

```
df['Dept'].mode()
```

```
df.mode()
```

min() / max() – Minimum and maximum values.

python

```
df['Age'].min()
```

```
df['Salary'].max()
```

sum() – Sum of values.

python

```
df['Sales'].sum()
```

```
df.sum()          # sum of all numeric columns
```

count() – Count non-null values.

python

```
df.count()        # non-null count per column
```

```
df['Age'].count()
```

std() – Standard deviation.

python

```
df['Salary'].std()
```

var() – Variance.

python

```
df['Salary'].var()
```

skew() – Skewness (asymmetry of distribution).

python

```
df['Income'].skew()
```

kurt() – Kurtosis (tail heaviness).

python

```
df['Income'].kurt()
```

corr() – Correlation between columns.

python

```
df.corr()
```

```
df['Age'].corr(df['Salary'])
```

cov() – Covariance between columns.

python

```
df.cov()
```

```
df['Age'].cov(df['Salary'])
```

RANKING & EXTREMES

rank() – Rank values within a Series.

python

```
df['Score'].rank()
```

```
df['Score'].rank(ascending=False, method='min')
```

nlargest() – Return n rows with largest values.

python

```
df.nlargest(5, 'Salary')
```

nsmallest() – Return n rows with smallest values.

python

```
df.nsmallest(3, 'Age')
```

idxmax() – Index of the maximum value.

python

```
df['Salary'].idxmax()      # row index where salary is max
```

idxmin() – Index of the minimum value.

python

```
df['Score'].idxmin()      # row index where score is min
```

EXPORTING DATA

to_csv() – Save DataFrame to a CSV file.

python

```
df.to_csv('output.csv', index=False)
```

to_excel() – Save to Excel file.

python

```
df.to_excel('output.xlsx', sheet_name='Sheet1', index=False)
```

to_json() – Convert to JSON format.

python

```
df.to_json('output.json', orient='records')
```

to_dict() – Convert to Python dictionary.

python

df.to_dict()

df.to_dict(orient='records') # list of row dicts

to_numpy() – Convert to NumPy array.

python

df.to_numpy()

df[['Age', 'Salary']].to_numpy()

COMPARISON

equals() – Check if two DataFrames are exactly equal.

python

df1.equals(df2) # True or False

RESHAPING

stack() – Pivot innermost column level to row index.

python

df.stack()

unstack() – Pivot innermost row index to column.

python

df.unstack()

melt() – Unpivot wide format to long format.

python

```
df.melt(id_vars=['Name'], value_vars=['Math', 'Science'], var_name='Subject',  
value_name='Score')
```

pivot() – Reshape using column values as new columns.

python

```
df.pivot(index='Date', columns='Category', values='Sales')
```

explode() – Transform list-like elements into separate rows.

python

```
df['Tags'].explode()
```

```
df.explode('Tags')
```

TIME-BASED OPERATIONS

shift() – Shift values up or down by n periods.

python

```
df['Sales'].shift(1)      # lag by 1
```

```
df['Sales'].shift(-1)     # lead by 1
```

rolling() – Rolling window calculations.

python

```
df['Sales'].rolling(window=7).mean()  # 7-day moving average
```

```
df['Price'].rolling(3).sum()
```

expanding() – Expanding window (grows with each row).

python

```
df['Sales'].expanding().mean()  # cumulative mean
```

cumsum() – Cumulative sum.

python

```
df['Sales'].cumsum()
```

cumprod() – Cumulative product.

python

```
df['Growth'].cumprod()
```

cummax() – Cumulative maximum.

python

```
df['Score'].cummax()
```

cummin() – Cumulative minimum.

python

```
df['Score'].cummin()
```

resample() – Resample time-series data to a new frequency.

python

```
df.resample('M').sum()      # monthly totals
```

```
df.resample('W').mean()    # weekly averages
```

asfreq() – Convert time-series to specified frequency.

python

```
df.asfreq('D')             # daily frequency
```

```
df.asfreq('H', fill_value=0) # hourly, fill gaps with 0
```

between_time() – Select rows between two times of day.

python

```
df.between_time('09:00', '17:00')
```

at_time() – Select rows at a specific time of day.

python

```
df.at_time('09:30')
```

tz_localize() – Assign a timezone to a naive DatetimeIndex.

python

```
df.tz_localize('UTC')
```

```
df.tz_localize('Asia/Kolkata')
```

tz_convert() – Convert timezone-aware DatetimeIndex to another timezone.

python

```
df.tz_convert('US/Eastern')
```

```
df.tz_convert('Asia/Kolkata')
```

DTYPE UTILITIES

select_dtypes() – Select columns by data type.

python

```
df.select_dtypes(include='number')    # numeric columns only
```

```
df.select_dtypes(exclude=['object'])  # exclude string columns
```

```
df.select_dtypes(include=['int64', 'float64'])
```

infer_objects() – Infer better dtypes for object columns.

python

```
df.infer_objects()
```

convert_dtypes() – Convert to best possible nullable dtypes.

python

```
df.convert_dtypes()    # auto-converts to Int64, StringDtype, etc.
```

